

Real-Time Examples of Open-Closed Principle in C#

In this article, I will discuss **Multiple Real-Time Examples of the Open-Closed Principle (OCP) in C#**. Please read our previous article discussing the basic concepts of the [Open-Closed Principle \(OCP\) in C#](#). At the end of this article, you will understand the following Real-time Examples using the Open-Closed Principle (OCP) in C#.

1. **E-commerce System that Provides Discounts**
2. **Notification System**
3. **Logging Mechanism**
4. **Tax Calculation System**
5. **Report Generation System**
6. **Payment Gateway System**
7. **Vehicle Service Center**
8. **Filtering Products Based on Different Criteria**
9. **Dynamic Survey Builder**
10. **Employee Management System**

The Open-Closed Principle (OCP) is one of the SOLID principles of object-oriented design. It states that software entities (like classes, modules, and functions) should be open for extension but closed for modification.

Note: First, I will show the example without following OCP. Then, I will discuss the problem of not following the Open-Closed Principle, and then we will write the same example following the Open-Closed Principle.

How to use the Open-Closed Principle in C#?

To apply the Open-Closed Principle (OCP) in C#, you must design your classes and modules to allow easy extension without modifying existing code. Here's a step-by-step guide on how to use the Open-Closed Principle effectively in C#:

- **Identify Areas of Change:** Identify the parts of your application that are likely to change or require new functionality in the future. These are the areas where you want to apply the OCP.
- **Use Abstraction:** Define abstract classes, interfaces, or base classes that provide a common contract or API for your building functionality. This contract represents the extension point for future changes.
- **Create Implementations:** Implement concrete classes that adhere to the abstraction. Each implementation represents a specific variant or extension of the functionality.
- **Design for Extensibility:** Ensure that the base classes, interfaces, or abstractions are designed to be extended. They should define methods or properties that allow subclasses to override or extend behavior.
- **Avoid Tight Coupling:** Avoid hardcoding dependencies on concrete classes in your code. Instead, rely on abstractions or interfaces. This way, you can easily swap out implementations without modifying consuming code.
- **Use Dependency Injection:** Apply dependency injection to inject instances of implementations into classes that need them. This enables you to change implementations without modifying client code.
- **Use Design Patterns:** Utilize design patterns like Strategy, Decorator, Factory Method, and Abstract Factory to achieve open-closed behavior effectively. These patterns provide guidance on structuring your code to accommodate future changes.
- **Document Extensions:** Document how new functionality can be added by extending existing abstractions or interfaces. This helps other developers understand how to adhere to the OCP when

making changes.

Real-Time Example of Open-Closed Principle in C#: E-commerce System that Provides Discounts

Let's understand the Open-Closed Principle (OCP) with a real-world example. Imagine you're building an e-commerce system that provides discounts to different types of customers.

Without OCP:

If you hard-code each discount type, you would end up with a switch or if-else chain, and each time you wanted to add a new customer type or discount rule, you'd modify the class. Let us see how we can implement the above example without following the Open-Closed Principle in C#:

```
using System;
namespace OCPDemo
{
    public enum CustomerType
    {
        Regular,
        Premium,
        Newbie
    }
    public class DiscountCalculator
    {
        public double CalculateDiscount(double price, CustomerType customerType)
        {
            switch (customerType)
            {
                case CustomerType.Regular:
                    return price * 0.1; // 10% discount for regular customers
                case CustomerType.Premium:
                    return price * 0.3; // 30% discount for premium customers
                case CustomerType.Newbie:
                    return price * 0.05; // 5% discount for new customers
                default:
                    throw new ArgumentOutOfRangeException();
            }
        }
    }
}
```

With OCP:

We can define a strategy pattern to adhere to OCP. Each discount type will have its own class, and adding a new discount would mean adding a new class without modifying the existing ones. Let us see how we can implement the above example following the Open-Closed Principle in C#:

```
using System;
```

```

namespace OCPDemo
{
    //Create an interface for the discount strategy
    public interface IDiscountStrategy
    {
        double CalculateDiscount(double price);
    }
    //Implement this interface for each discount type
    public class RegularDiscount : IDiscountStrategy
    {
        public double CalculateDiscount(double price)
        {
            return price * 0.1;
        }
    }
    public class PremiumDiscount : IDiscountStrategy
    {
        public double CalculateDiscount(double price)
        {
            return price * 0.3;
        }
    }
    public class NewbieDiscount : IDiscountStrategy
    {
        public double CalculateDiscount(double price)
        {
            return price * 0.05;
        }
    }
    //Modify the DiscountCalculator class to accept an IDiscountStrategy
    public class DiscountCalculator
    {
        private readonly IDiscountStrategy _discountStrategy;
        public DiscountCalculator(IDiscountStrategy discountStrategy)
        {
            _discountStrategy = discountStrategy;
        }
        public double CalculateDiscount(double price)
        {
            return _discountStrategy.CalculateDiscount(price);
        }
    }

    //Testing the Open-Closed Principle
    public class Program
    {
        public static void Main()
        {
            var regularDiscount = new RegularDiscount();
            var calculator = new DiscountCalculator(regularDiscount);
        }
    }
}

```

```

    double discountedPrice = calculator.CalculateDiscount(100); // 10%
    applied
    var premiumDiscount = new PremiumDiscount();
    calculator = new DiscountCalculator(premiumDiscount);
    discountedPrice = calculator.CalculateDiscount(100); // 30% discount
    Console.ReadKey();
}
}
}

```

In this approach, if a new discount strategy needs to be introduced, you create a new class implementing the `IDiscountStrategy` without modifying the existing system. This aligns with OCP.

Real-Time Example of Open-Closed Principle in C#: Notification System

Let's take an example of a notification system where a user can be notified through various channels.

Without OCP:

Imagine if you hard-code each notification channel, you'd likely use a switch or if-else statement, and every time a new channel needs to be added, the `NotificationSender` class would be modified. Let us see how we can implement the above example without following the Open-Closed Principle in C#:

```

using System;
namespace OCPDemo
{
    public enum NotificationChannel
    {
        Email,
        SMS
    }
    public class NotificationSender
    {
        public void SendNotification(NotificationChannel channel, string message)
        {
            switch (channel)
            {
                case NotificationChannel.Email:
                    Console.WriteLine($"Sending Email: {message}");
                    break;
                case NotificationChannel.SMS:
                    Console.WriteLine($"Sending SMS: {message}");
                    break;
                default:
                    throw new ArgumentOutOfRangeException();
            }
        }
    }
}

```

With OCP:

We'll use a strategy pattern for each notification channel to adhere to the Open-Closed Principle.

```
using System;
namespace OCPDemo
{
    //Start with an interface
    public interface INotificationChannel
    {
        void Send(string message);
    }
    //Implement this interface for each channel
    public class EmailNotification : INotificationChannel
    {
        public void Send(string message)
        {
            Console.WriteLine($"Sending Email: {message}");
        }
    }
    public class SMSNotification : INotificationChannel
    {
        public void Send(string message)
        {
            Console.WriteLine($"Sending SMS: {message}");
        }
    }
    //Modify the NotificationSender class to accept any INotificationChannel
    public class NotificationSender
    {
        private readonly INotificationChannel _channel;
        public NotificationSender(INotificationChannel channel)
        {
            _channel = channel;
        }
        public void SendNotification(string message)
        {
            _channel.Send(message);
        }
    }

    //Testing the Open-Closed Principle
    public class Program
    {
        public static void Main()
        {
            var emailChannel = new EmailNotification();
            var sender = new NotificationSender(emailChannel);
            sender.SendNotification("Hello via Email!");
            var smsChannel = new SMSNotification();
        }
    }
}
```

```

        sender = new NotificationSender(smsChannel);
        sender.SendNotification("Hello via SMS!");
        Console.ReadKey();
    }
}
}

```

In the future, if you'd like to introduce a Slack channel, you'd create a `SlackNotification` class that implements `INotificationChannel` without modifying the `NotificationSender` class. This demonstrates how a system can be open for extensions (new channels) but closed for modifications. When you run the above code, you will get the following output.

Real-Time Example of Open-Closed Principle in C#: Logging Mechanism

Let's understand the Open-Closed Principle (OCP) with a real-world example of a logging mechanism. Suppose you are building a system where different logs (like errors, warnings, and info) must be recorded. These logs can be saved in various destinations, such as a file, a database, or the cloud.

Without OCP:

Let us see how we can implement the above example without following the Open-Closed Principle in C#:

```

using System;
namespace OCPDemo
{
    public enum LogType
    {
        File,
        Database
    }
    public class Logger
    {
        public void LogMessage(string message, LogType type)
        {
            switch (type)
            {
                case LogType.File:
                    // Logic to write the message to a file
                    Console.WriteLine($"Logged to file: {message}");
                    break;
                case LogType.Database:
                    // Logic to write the message to a database
                    Console.WriteLine($"Logged to database: {message}");
                    break;
                default:
                    throw new ArgumentException("Invalid log type");
            }
        }
    }
}

```

```

    }
}
}

```

With the above design, you'd need to modify the Logger class if you add another logging mechanism, e.g., cloud storage. This approach violates the OCP.

With OCP:

```

using System;
namespace OCPDemo
{
    //Define an interface for logging
    public interface ILogger
    {
        void LogMessage(string message);
    }
    //Implement the interface for each logging type
    public class FileLogger : ILogger
    {
        public void LogMessage(string message)
        {
            // Logic to write the message to a file
            Console.WriteLine($"Logged to file: {message}");
        }
    }
    public class DatabaseLogger : ILogger
    {
        public void LogMessage(string message)
        {
            // Logic to write the message to a database
            Console.WriteLine($"Logged to database: {message}");
        }
    }
    //Modify the main Logger class to work with any logging type
    public class Logger
    {
        private readonly ILogger _logger;
        public Logger(ILogger logger)
        {
            _logger = logger;
        }
        public void LogMessage(string message)
        {
            _logger.LogMessage(message);
        }
    }

    //Testing the Open-Closed Principle

```

```

public class Program
{
    public static void Main()
    {
        var fileLogger = new FileLogger();
        var logger = new Logger(fileLogger);
        logger.LogMessage("This is a file log.");
        var dbLogger = new DatabaseLogger();
        logger = new Logger(dbLogger);
        logger.LogMessage("This is a database log.");

        Console.ReadKey();
    }
}

```

In the future, to add a new logging destination, like cloud storage, you implement the ILogger interface in a new class, CloudLogger, without needing to modify the existing Logger class. This fulfills the essence of OCP: open for extension (adding new logging mechanisms) but closed for modification (no changes to the Logger class). When you run the above code, you will get the following output.

Real-Time Example of Open-Closed Principle in C#: Tax Calculation System

Let's understand another real-time example, i.e., the Tax Calculation System using the Open-Closed Principle (OCP). Imagine you are building a system for an online retail store where different products have different tax rates based on their category. Over time, new categories of products will be introduced, and tax rules might change or be added.

Without OCP:

Let us see how we can implement the above example without following the Open-Closed Principle in C#. If you hard-code the tax calculation for each category, the structure might look like this:

```

using System;
namespace OCPDemo
{
    public enum ProductCategory
    {
        Electronics,
        Groceries,
        Clothing
    }
    public class Product
    {
        public string Name { get; set; }
        public double Price { get; set; }
    }
}

```

```

        public ProductCategory Category { get; set; }
    }
    public class TaxCalculator
    {
        public double CalculateTax(Product product)
        {
            switch (product.Category)
            {
                case ProductCategory.Electronics:
                    return product.Price * 0.15; // 15% tax
                case ProductCategory.Groceries:
                    return product.Price * 0.05; // 5% tax
                case ProductCategory.Clothing:
                    return product.Price * 0.10; // 10% tax
                default:
                    throw new ArgumentException("Unsupported product category")
            }
        }
    }
}

```

This design is not ideal. You'd have to modify the CalculateTax method for every new category or tax rule.

With OCP:

Let us see how we can implement the above example following the Open-Closed Principle in C#:

```

using System;
namespace OCPDemo
{
    public enum ProductCategory
    {
        Electronics,
        Groceries,
        Clothing
    }
    public class Product
    {
        public string Name { get; set; }
        public double Price { get; set; }
        public ProductCategory Category { get; set; }
    }
    //Define an interface for tax calculation
    public interface ITaxStrategy
    {
        double CalculateTax(Product product);
    }
    //Implement this interface for each category
    public class ElectronicsTaxStrategy : ITaxStrategy
    {

```

```

        public double CalculateTax(Product product)
        {
            return product.Price * 0.15;
        }
    }
    public class GroceriesTaxStrategy : ITaxStrategy
    {
        public double CalculateTax(Product product)
        {
            return product.Price * 0.05;
        }
    }
    public class ClothingTaxStrategy : ITaxStrategy
    {
        public double CalculateTax(Product product)
        {
            return product.Price * 0.10;
        }
    }
    //Refactor the main tax calculator to utilize these strategies
    public class TaxCalculator
    {
        private readonly ITaxStrategy _taxStrategy;
        public TaxCalculator(ITaxStrategy taxStrategy)
        {
            _taxStrategy = taxStrategy;
        }
        public double CalculateTax(Product product)
        {
            return _taxStrategy.CalculateTax(product);
        }
    }

    //Testing the Open-Closed Principle
    public class Program
    {
        public static void Main()
        {
            var electronicsProduct = new Product { Name = "Laptop", Price = 1000,
Category = ProductCategory.Electronics };
            var taxCalculator = new TaxCalculator(new ElectronicsTaxStrategy());
            Console.WriteLine($"Tax for {electronicsProduct.Name}:
${taxCalculator.CalculateTax(electronicsProduct)}");
            Console.ReadKey();
        }
    }
}

```

With this approach, when introducing a new product category or tax rules change, you only need to create a new tax strategy class. There's no need to modify the existing TaxCalculator or other strategy classes, which aligns with the Open-Closed Principle: the code remains open for extension and closes for modification.

Real-Time Example of Open-Closed Principle in C#: Report Generation System

Let's understand the real-world example of the Open-Closed Principle (OCP) involving a report generation system. Suppose you're developing a reporting system where users can generate reports in formats like PDF, CSV, and XML. Over time, you anticipate the need to support more formats.

Without OCP:

If you hard-code the report generation for each format, you might do something like:

```
using System;
namespace OCPDemo
{
    public enum ReportType
    {
        PDF,
        CSV,
        XML
    }
    public class ReportGenerator
    {
        public void GenerateReport(string data, ReportType type)
        {
            switch (type)
            {
                case ReportType.PDF:
                    Console.WriteLine($"Generating PDF report with data: {data}");
                    break;
                case ReportType.CSV:
                    Console.WriteLine($"Generating CSV report with data: {data}");
                    break;
                case ReportType.XML:
                    Console.WriteLine($"Generating XML report with data: {data}");
                    break;
                default:
                    throw new ArgumentException("Invalid report type");
            }
        }
    }
}
```

If you needed to add another format in the future, like JSON, you'd have to modify the ReportGenerator class, which violates OCP.

With OCP:

Let us see how we can implement the above example following the Open-Closed Principle in C#:

```
using System;
```

```

namespace OCPDemo
{
    //Define an interface for report generation
    public interface IReportGenerator
    {
        void GenerateReport(string data);
    }
    //Implement this interface for each report type
    public class PDFReportGenerator : IReportGenerator
    {
        public void GenerateReport(string data)
        {
            Console.WriteLine($"Generating PDF report with data: {data}");
        }
    }
    public class CSVReportGenerator : IReportGenerator
    {
        public void GenerateReport(string data)
        {
            Console.WriteLine($"Generating CSV report with data: {data}");
        }
    }
    public class XMLReportGenerator : IReportGenerator
    {
        public void GenerateReport(string data)
        {
            Console.WriteLine($"Generating XML report with data: {data}");
        }
    }
    //Modify the main ReportGenerationService to work with any report generator
    public class ReportGenerationService
    {
        private readonly IReportGenerator _reportGenerator;
        public ReportGenerationService(IReportGenerator reportGenerator)
        {
            _reportGenerator = reportGenerator;
        }
        public void GenerateReport(string data)
        {
            _reportGenerator.GenerateReport(data);
        }
    }

    //Testing the Open-Closed Principle
    public class Program
    {
        public static void Main()
        {
            var pdfGenerator = new PDFReportGenerator();
            var service = new ReportGenerationService(pdfGenerator);
            service.GenerateReport("PDF Report Data");
        }
    }
}

```

```

        var csvGenerator = new CSVReportGenerator();
        service = new ReportGenerationService(csvGenerator);
        service.GenerateReport("CSV Report Data");

        Console.ReadKey();
    }
}

```

If you need to introduce a new format, e.g., JSON, you will create a `JSONReportGenerator` class that implements `IReportGenerator`. No need to modify the existing `ReportGenerationService` class or other report generator classes. This approach is open for extension (adding new report types) but closed for modification, adhering to the OCP. When you run the above code, you will get the following output.

Real-Time Example of Open-Closed Principle in C#: Payment Gateway System

Let's understand the real-world example of the Open-Closed Principle (OCP) involving a Payment Gateway System. Imagine you're building an e-commerce application where customers can make payments using various methods: credit cards, PayPal, or bank transfers. As the e-commerce industry evolves, you anticipate supporting more payment methods.

Without OCP:

If you use conditional logic to handle each payment type, the code might look something like this:

```

using System;
namespace OCPDemo
{
    public enum PaymentMethod
    {
        CreditCard,
        PayPal,
        BankTransfer
    }
    public class PaymentProcessor
    {
        public void ProcessPayment(PaymentMethod method, double amount)
        {
            switch (method)
            {
                case PaymentMethod.CreditCard:
                    Console.WriteLine($"Processing credit card payment for amount
                    ${amount}");
                    break;
                case PaymentMethod.PayPal:

```

```

        Console.WriteLine($"Processing PayPal payment for amount:
        ${amount}");
        break;
    case PaymentMethod.BankTransfer:
        Console.WriteLine($"Processing bank transfer for amount: $
        ${amount}");
        break;
    default:
        throw new ArgumentException("Unsupported payment method");
    }
}
}
}
}

```

The problem with this design is that each time a new payment method is added, you'll have to modify the PaymentProcessor class.

With OCP:

Let us see how we can implement the above example following the Open-Closed Principle in C#:

```

using System;
namespace OCPDemo
{
    //Define an interface for processing payments
    public interface IPaymentProcessor
    {
        void ProcessPayment(double amount);
    }
    //Implement this interface for each payment method
    public class CreditCardPayment : IPaymentProcessor
    {
        public void ProcessPayment(double amount)
        {
            Console.WriteLine($"Processing credit card payment for amount: ${amount}");
        }
    }
    public class PayPalPayment : IPaymentProcessor
    {
        public void ProcessPayment(double amount)
        {
            Console.WriteLine($"Processing PayPal payment for amount: ${amount}");
        }
    }
    public class BankTransferPayment : IPaymentProcessor
    {
        public void ProcessPayment(double amount)
        {
            Console.WriteLine($"Processing bank transfer for amount: ${amount}");
        }
    }
}

```

```
//Modify the main payment handler
public class PaymentHandler
{
    private readonly IPaymentProcessor _paymentProcessor;
    public PaymentHandler(IPaymentProcessor paymentProcessor)
    {
        _paymentProcessor = paymentProcessor;
    }
    public void ExecutePayment(double amount)
    {
        _paymentProcessor.ProcessPayment(amount);
    }
}

//Testing the Open-Closed Principle
public class Program
{
    public static void Main()
    {
        var creditCardPayment = new CreditCardPayment();
        var handler = new PaymentHandler(creditCardPayment);
        handler.ExecutePayment(100.50);
        var paypalPayment = new PayPalPayment();
        handler = new PaymentHandler(paypalPayment);
        handler.ExecutePayment(150.75);

        Console.ReadKey();
    }
}
}
```

With this design, when you need to add a new payment method (e.g., a crypto payment), you'd create a new class (e.g., CryptoPayment) that implements IPaymentProcessor. There's no need to modify the existing PaymentHandler or other payment classes. This design is open for extension but closed for modification, in line with the OCP.

Real-Time Example of Open-Closed Principle in C#: Vehicle Service Center

Let's understand another real-world inspired example involving a vehicle service center that caters to various vehicle types. Suppose you are developing a system for a vehicle service center where different types of vehicles (like cars, trucks, motorcycles) can be serviced. Each type might have a distinct servicing procedure. You anticipate the addition of more vehicle types in the future.

Without OCP:

Let us see how we can implement the above example without following the Open-Closed Principle in C#. Embedding the servicing logic for each vehicle type directly in a method would look something like:

```
using System;
namespace OCPDemo
```

```

{
    public enum VehicleType
    {
        Car,
        Truck,
        Motorcycle
    }
    public class ServiceCenter
    {
        public void ServiceVehicle(VehicleType vehicle, string model)
        {
            switch (vehicle)
            {
                case VehicleType.Car:
                    Console.WriteLine($"Servicing Car. Model: {model}");
                    break;
                case VehicleType.Truck:
                    Console.WriteLine($"Servicing Truck. Model: {model}");
                    break;
                case VehicleType.Motorcycle:
                    Console.WriteLine($"Servicing Motorcycle. Model: {model}");
                    break;
                default:
                    throw new ArgumentException("Unsupported vehicle type");
            }
        }
    }
}

```

The major downside is that the ServiceVehicle method would require modification for every new vehicle type.

With OCP:

Let us see how we can implement the above example following the Open-Closed Principle in C#:

```

using System;
namespace OCPDemo
{
    //Create an interface for vehicle service
    public interface IVehicleService
    {
        void Service(string model);
    }
    //Implement the interface for each vehicle type
    public class CarService : IVehicleService
    {
        public void Service(string model)
        {
            Console.WriteLine($"Servicing Car. Model: {model}");
        }
    }
}

```

```

}
public class TruckService : IVehicleService
{
    public void Service(string model)
    {
        Console.WriteLine($"Servicing Truck. Model: {model}");
    }
}
public class MotorcycleService : IVehicleService
{
    public void Service(string model)
    {
        Console.WriteLine($"Servicing Motorcycle. Model: {model}");
    }
}
//Update the Service Center to accept any vehicle service
public class ServiceCenter
{
    private readonly IVehicleService _vehicleService;
    public ServiceCenter(IVehicleService vehicleService)
    {
        _vehicleService = vehicleService;
    }
    public void ServiceVehicle(string model)
    {
        _vehicleService.Service(model);
    }
}

//Testing the Open-Closed Principle
public class Program
{
    public static void Main()
    {
        var carService = new CarService();
        var center = new ServiceCenter(carService);
        center.ServiceVehicle("Sedan");
        var truckService = new TruckService();
        center = new ServiceCenter(truckService);
        center.ServiceVehicle("18-Wheeler");

        Console.ReadKey();
    }
}
}

```

When the service center decides to cater to a new vehicle type (like a bicycle), all that's needed is a new class (e.g., BicycleService) implementing the IVehicleService interface. The core ServiceCenter class remains untouched, embodying the spirit of the Open-Closed Principle: open for extension but closed for modification.

Real-Time Example of Open-Closed Principle in C#: Filtering Products Based on Different Criteria

Let's explore the Open-Closed Principle (OCP) through another real-world example of filtering products based on different criteria. Suppose you have an e-commerce platform where users can view a list of products. As the platform grows, the need arises to filter these products based on different criteria: color, size, brand, etc.

Without OCP:

Let us see how we can implement the above example without following the Open-Closed Principle in C#. You might implement filtering with a series of methods for each criteria:

```
using System.Collections.Generic;
using System.Linq;
namespace OCPDemo
{
    public class Product
    {
        public string Name { get; set; }
        public string Color { get; set; }
        public double Size { get; set; }
        // ... other properties
    }
    public class ProductFilter
    {
        public IEnumerable<Product> FilterByColor(IEnumerable<Product> products,
color)
        {
            return products.Where(p => p.Color == color);
        }
        public IEnumerable<Product> FilterBySize(IEnumerable<Product> products,
size)
        {
            return products.Where(p => p.Size == size);
        }
        // For every new criteria, you add a new method
    }
}
```

This approach becomes cumbersome as the number of filter criteria grows. For instance, if you wanted to filter by color and size, you'd end up with a combinatorial explosion of methods.

With OCP:

Let us see how we can implement the above example following the Open-Closed Principle in C#:

```
using System;
using System.Collections.Generic;
```

```

using System.Linq;
namespace OCPDemo
{
    public class Product
    {
        public string Name { get; set; }
        public string Color { get; set; }
        public double Size { get; set; }
        // ... other properties
    }
    //Implement a specification pattern
    public interface ISpecification<T>
    {
        bool IsSatisfied(T item);
    }
    public interface IFilter<T>
    {
        IEnumerable<T> Filter(IEnumerable<T> items, ISpecification<T> spec);
    }
    //Define specifications for each criteria
    public class ColorSpecification : ISpecification<Product>
    {
        private readonly string _color;
        public ColorSpecification(string color)
        {
            _color = color;
        }
        public bool IsSatisfied(Product p)
        {
            return p.Color == _color;
        }
    }
    public class SizeSpecification : ISpecification<Product>
    {
        private readonly double _size;
        public SizeSpecification(double size)
        {
            _size = size;
        }
        public bool IsSatisfied(Product p)
        {
            return p.Size == _size;
        }
    }
    //Implement the filter
    public class ProductFilter : IFilter<Product>
    {
        public IEnumerable<Product> Filter(IEnumerable<Product> items,
ISpecification<Product> spec)
        {
            return items.Where(p => spec.IsSatisfied(p));
        }
    }
}

```

```

    }
}

//Testing the Open-Closed Principle
public class Program
{
    public static void Main()
    {
        var products = new List<Product>
        {
            new Product { Name = "Red Shirt", Color = "Red", Size = 10 },
            new Product { Name = "Blue Pants", Color = "Blue", Size = 20 }
            // ... other products
        };
        var filter = new ProductFilter();
        foreach (var product in filter.Filter(products, new
ColorSpecification("Red")))
        {
            Console.WriteLine($"- {product.Name}");
        }

        Console.ReadKey();
    }
}
}

```

If you want to add more filter criteria, you create new specification classes without modifying the existing ProductFilter class. This architecture adheres to OCP and keeps the code modular and extensible.

Real-Time Example of Open-Closed Principle in C#: Dynamic Survey Builder

Let's understand the Open-Closed Principle (OCP) through another real-world example, Dynamic Survey Builder. Suppose you have a web application that allows users to build surveys. There are multiple types of questions: multiple choice, true/false, and open-ended text. But you anticipate the possibility of adding more question types in the future, like rating scale, date selection, etc.

Without OCP:

Let us see how we can implement the above example without following the Open-Closed Principle in C#. You might have a method in your survey class to render each question type:

```

using System;
namespace OCPDemo
{
    public enum QuestionType
    {
        MultipleChoice,
        TrueFalse,
        OpenText
    }
}

```

```

public class Question
{
    public string Title { get; set; }
    public QuestionType Type { get; set; }
    // ... other properties related to the question
}
public class SurveyRenderer
{
    public void RenderQuestion(Question question)
    {
        switch (question.Type)
        {
            case QuestionType.MultipleChoice:
                Console.WriteLine($"Rendering multiple choice question:
{question.Title}");
                break;
            case QuestionType.TrueFalse:
                Console.WriteLine($"Rendering true/false question:
{question.Title}");
                break;
            case QuestionType.OpenText:
                Console.WriteLine($"Rendering open text question: {question.Title}");
                break;
            default:
                throw new ArgumentException("Unsupported question type");
        }
    }
}

```

This approach is problematic. You would have to modify the RenderQuestion method for every new question type.

With OCP:

Let us see how we can implement the above example following the Open-Closed Principle in C#:

```

using System;
namespace OCPDemo
{
    public enum QuestionType
    {
        MultipleChoice,
        TrueFalse,
        OpenText
    }
    public class Question
    {
        public string Title { get; set; }
        public QuestionType Type { get; set; }
    }
}

```

```

        // ... other properties related to the question
    }
    //Define an interface for rendering questions
    public interface IQuestionRenderer
    {
        void Render(Question question);
    }
    //Implement this interface for each question type
    public class MultipleChoiceRenderer : IQuestionRenderer
    {
        public void Render(Question question)
        {
            Console.WriteLine($"Rendering multiple choice question: {question.Title}");
            // Additional logic specific to multiple choice rendering
        }
    }
    public class TrueFalseRenderer : IQuestionRenderer
    {
        public void Render(Question question)
        {
            Console.WriteLine($"Rendering true/false question: {question.Title}");
            // Additional logic specific to true/false rendering
        }
    }
    public class OpenTextRenderer : IQuestionRenderer
    {
        public void Render(Question question)
        {
            Console.WriteLine($"Rendering open text question: {question.Title}");
            // Additional logic specific to open text rendering
        }
    }
    //Update the main survey renderer
    public class SurveyRenderer
    {
        private readonly IQuestionRenderer _renderer;
        public SurveyRenderer(IQuestionRenderer renderer)
        {
            _renderer = renderer;
        }
        public void RenderQuestion(Question question)
        {
            _renderer.Render(question);
        }
    }

    //Testing the Open-Closed Principle
    public class Program
    {
        public static void Main()
        {

```

```

        var mcQuestion = new Question { Title = "Choose a color", Type =
QuestionType.MultipleChoice };
        var renderer = new SurveyRenderer(new MultipleChoiceRenderer());
        renderer.RenderQuestion(mcQuestion);
        var tfQuestion = new Question { Title = "The sky is blue", Type =
QuestionType.TrueFalse };
        renderer = new SurveyRenderer(new TrueFalseRenderer());
        renderer.RenderQuestion(tfQuestion);

        Console.ReadKey();
    }
}
}

```

When a new question type, e.g., rating scale, needs to be added, you would create a new class, `RatingScaleRenderer`, that implements `IQuestionRenderer`. No need to modify the existing `SurveyRenderer` class or other rendering classes. This design adheres to the essence of OCP: open for extension but closed for modification.

Real-Time Example of Open-Closed Principle in C#: Employee Management System

Let's understand another real-time example, i.e., an Employee Management System using the Open-Closed Principle (OCP). Imagine you're developing a system for managing employee salaries. Initially, the company only had full-time employees. However, the company hires part-time employees and interns as the business evolves. Each type of employee has a different way of calculating their monthly salary.

Without OCP:

Let us see how we can implement the above example without following the Open-Closed Principle in C#. A simple approach might involve hard-coding the salary computation logic based on the employee type.

```

using System;
namespace OCPDemo
{
    public enum EmployeeType
    {
        FullTime,
        PartTime,
        Intern
    }
    public class Employee
    {
        public EmployeeType Type { get; set; }
        public double HourlyRate { get; set; }
        public double HoursWorked { get; set; }
    }
    public class SalaryCalculator
    {
        public double CalculateSalary(Employee employee)
        {

```

```

        switch (employee.Type)
        {
            case EmployeeType.FullTime:
                return employee.HourlyRate * 160; // Assuming 160 work hours per month

            case EmployeeType.PartTime:
                return employee.HourlyRate * employee.HoursWorked;
            case EmployeeType.Intern:
                return employee.HourlyRate * employee.HoursWorked * 0.5; // get 50% of their computed salary
            default:
                throw new ArgumentException("Invalid employee type");
        }
    }
}

```

Adding a new employee type would require changes to the CalculateSalary method, violating OCP.

With OCP:

Let us see how we can implement the above example following the Open-Closed Principle in C#:

```

using System;
namespace OCPDemo
{
    //Define an abstract class or interface for employee salary calculation
    public interface IEmployee
    {
        double CalculateSalary();
    }
    //Implement this interface for each employee type
    public class FullTimeEmployee : IEmployee
    {
        public double HourlyRate { get; set; }
        public double CalculateSalary()
        {
            return HourlyRate * 160;
        }
    }
    public class PartTimeEmployee : IEmployee
    {
        public double HourlyRate { get; set; }
        public double HoursWorked { get; set; }
        public double CalculateSalary()
        {
            return HourlyRate * HoursWorked;
        }
    }
    public class Intern : IEmployee

```

```

{
    public double HourlyRate { get; set; }
    public double HoursWorked { get; set; }
    public double CalculateSalary()
    {
        return HourlyRate * HoursWorked * 0.5;
    }
}
//Create a simple salary calculator that works for any employee type
public class SalaryCalculator
{
    public double CalculateSalary(IEmployee employee)
    {
        return employee.CalculateSalary();
    }
}

//Testing the Open-Closed Principle
public class Program
{
    public static void Main()
    {
        var fullTime = new FullTimeEmployee { HourlyRate = 25 };
        var salaryCalculator = new SalaryCalculator();
        Console.WriteLine($"Full-Time Employee Salary:
${salaryCalculator.CalculateSalary(fullTime)}");
        var intern = new Intern { HourlyRate = 20, HoursWorked = 80 };
        Console.WriteLine($"Intern Salary:
${salaryCalculator.CalculateSalary(intern)}");

        Console.ReadKey();
    }
}

```

If a new type of employee or a new salary calculation method is introduced, you will create a new class implementing IEmployee without modifying existing classes. This aligns with the Open-Closed Principle: the code is open for extension but closed for modification.

In the next article, I will discuss the [Liskov Substitution Principle in C#](#) with Examples. In this article, I explain **Real-Time Examples of the Open-Closed Principle (OCP) in C#**. I hope you enjoy this Open-Closed Principle (OCP) Real-Time Examples using the C# article